



UNICAMP

Relatório do Trabalho 2: Realce de Imagens Digitais

Relatório desenvolvido a partir da aplicação de procedimentos de realce em imagens digitais apresentados nas aulas de MC920 Introdução ao Processamento Digital de Imagem

Prof. Dr. Hélio Pedrini

Letícia Silva Brito
240314

Campinas, 7 de abril de 2025.

1. Introdução	2
2. Materiais	2
2.1. Requerimentos	2
2.2. Organização das entradas e das saídas	2
2.2.1. Entrada	3
2.2.2. Saída	3
3. Código	3
3.1. main.py	3
3.2. utils.py	3
3.3. ditherer.py	3
3.3.1. Floyd e Steinberg	4
3.3.2. Stevenson e Arce	5
3.3.3. Burkes	5
3.3.4. Sierra	6
3.3.5. Stucki	6
3.3.6. Jarvis, Judice e Ninke	7
3.4. frequency_domain.py	8
3.4.1. Espectro de Fourier	8
3.4.2. Aplicação dos filtros	8
3.5. compression.py	11

1. Introdução

Este trabalho foi elaborado aplicando técnicas de meios-tons e filtragem no domínio de frequência.

Os códigos que realizaram tais operações, assim como as imagens de entrada e saída, são encontrados no repositório Github: <https://github.com/Asteria101/MC920/tree/main/T-02>.

2. Materiais

Esta seção abordará o que é necessário para a execução e organização do código, de modo a tratar, além do código, a organização das entradas e saídas.

Fragmentou-se o código principal em diversos arquivos os quais contém as operações requeridas para imagens digitais, abaixo detalhou-se mais sobre cada um deles ao passo que analisa cada procedimento ou item do enunciado.

2.1. Requerimentos

Este projeto foi desenvolvido na linguagem de programação *Python* na versão 3.12.3, utilizando as seguintes bibliotecas (instaladas em um ambiente virtual, através do comando `pip install ...`), que possibilitaram a criação de algoritmos eficazes para trabalhar com imagens digitais:

Tabela 1: bibliotecas utilizadas e suas respectivas versões

Biblioteca	Versão
Scikit Image	0.25.2
NumPy	2.2.4
Matplotlib	3.10.1

A biblioteca *Scikit Image* foi utilizada para realizar algumas operações mais específicas com as imagens, assim como as básicas, como ler a imagem em um vetor *NumPy*. Assim, a biblioteca *NumPy* serve para realizar operações de Álgebra Linear e Geometria Analítica com as imagens digitais. Por fim, *Matplotlib* é usada para visualizar e salvar as imagens na máquina.

2.2. Organização das entradas e das saídas

Abaixo detalhou-se como foi feita a ordenação e separação das imagens de entrada e saída de modo a facilitar a chamada e visualização de tais arquivos no diretório.

2.2.1. Entrada

Utilizou-se 3 (três) arquivos .png que foram usados de entrada se encontram no diretório *in-images*. As imagens provêm do [diretório](#) de imagens fornecidas pelo Professor Hélio Pedrini. Há uma imagem monocromática (*baboon_mono.png*) e duas coloridas (*baboon.png* e *peppers.png*).

2.2.2. Saída

As saídas encontram-se no diretório *out-images* e estão divididas conforme os itens do enunciado, de tal maneira que há: *1.1* e *1.2*. A primeira pasta contém as saídas para as técnicas de meios-tons, de modo que *c_* e *m_* ao início de cada arquivo indica, respectivamente, colorido e monocromático; enquanto a segunda, dedica-se para a filtragem no domínio de frequência e está dividida por *peppers* e *baboon*.

3. Código

A boa estruturação de um programa gera uma maior eficiência e facilidade de manutenção, assim, optou-se por dividir o código em alguns arquivos. Abaixo, há a explanação de cada um contendo a análise do item proposto.

3.1. main.py

Contém a chamada de todos os arquivos para realização de suas funções assim como o armazenamento das imagens em seus respectivos lugares.

3.2. utils.py

Este arquivo contém as importações de bibliotecas necessárias para o funcionamento dos códigos, assim como três funções gerais: *show(img : np.ndarray)*, *save(img : np.ndarray, filepath : str)* e *histogramIt(img : np.ndarray)*. A primeira é responsável por mostrar a imagem na tela, a segunda por salvar a imagem na máquina e a terceira produz um histograma da imagem.

3.3. ditherer.py

Este arquivo contém a função *dither(img : np.ndarray, technic : list[list], m : int, n : int, traverse : int)*, que é responsável por aplicar os filtros de difusão de erro apresentados no enunciado do Trabalho 2. Basicamente, recebe-se uma imagem; a matriz da técnica a ser utilizada, em que cada linha apresenta o erro e os *offsets* de coluna e linha onde esse erro deve ser propagado (isto é, *[x_offset, y_offset, error]*); as dimensões (*width x height*) da matriz que representa a técnica em questão, onde *m = width - 1* e *n = height - 1*; e, por fim, qual o tipo de percurso a ser realizado. Neste trabalho, realizou-se apenas um percurso: alternado ou zigue-zague.

Assim, ao aplicar em uma imagem colorida (Imagem 1) e em uma monocromática (Imagem 2), pode-se observar os seguintes resultados para cada técnica de meios-tons.

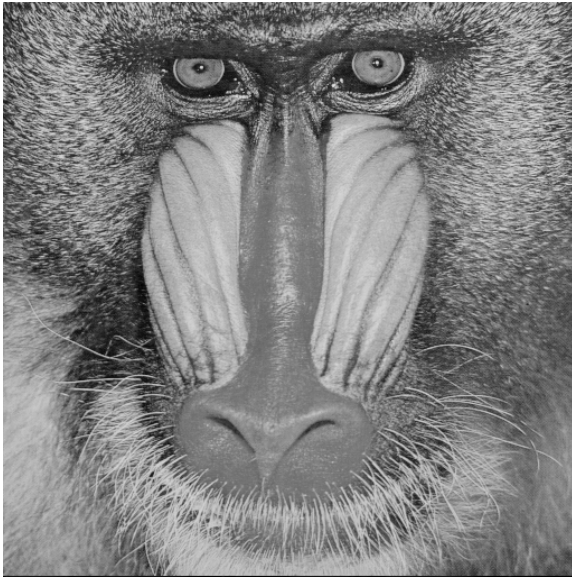


Imagem 1: baboon_mono.png

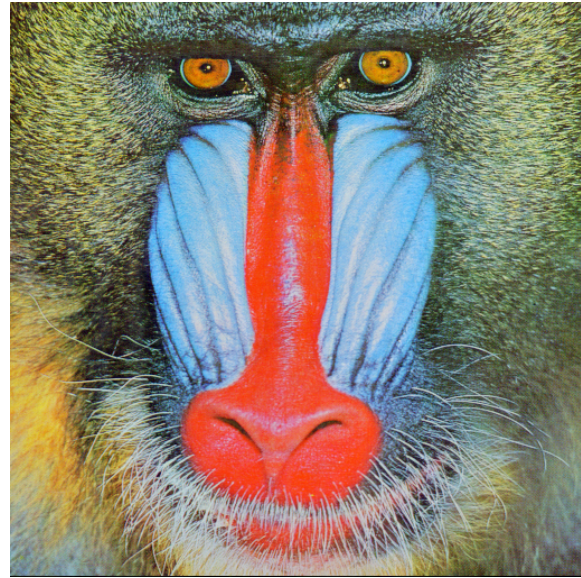


Imagem 1.1: baboon.png (baboon_mono.png colorido)

3.3.1. Floyd e Steinberg

Esta técnica de difusão de erro consegue manter uma qualidade razoável das imagens de modo que, muitos detalhes ainda conseguem ser preservados.

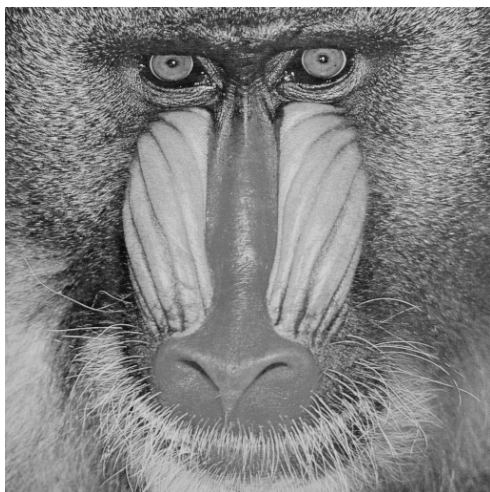


Imagem 1: imagem original (512x512)

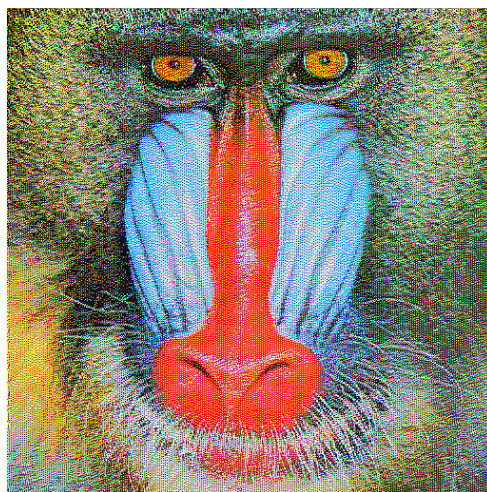


Imagem 2: difusão de Floyd-Steinberg aplicado na banda de cor

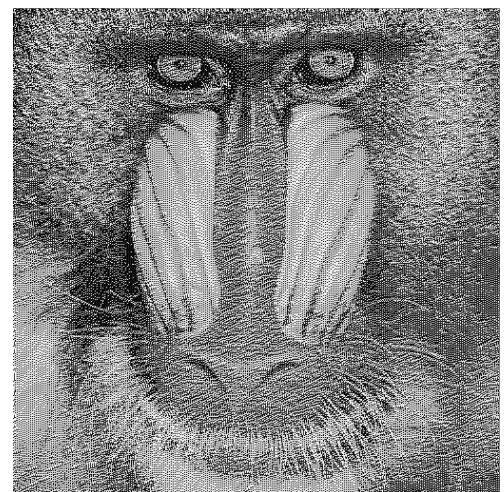


Imagem 3: difusão de Floyd-Steinberg aplicado na banda monocromática

3.3.2. Stevenson e Arce

Esta técnica aparenta funcionar melhor para imagens coloridas, visto que, para as monocromáticas os padrões gerados pela difusão marcam a imagem dependendo da ampliação da mesma.

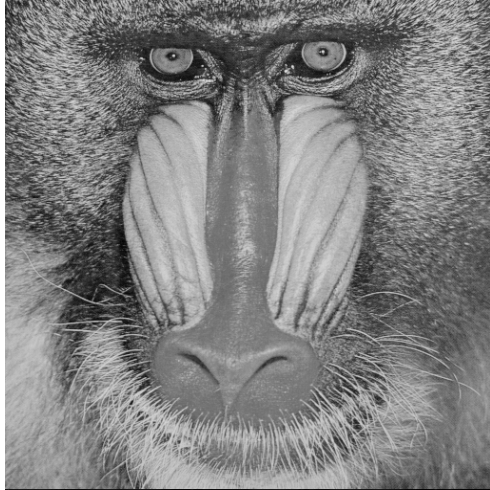


Imagem 1: imagem original (512x512)

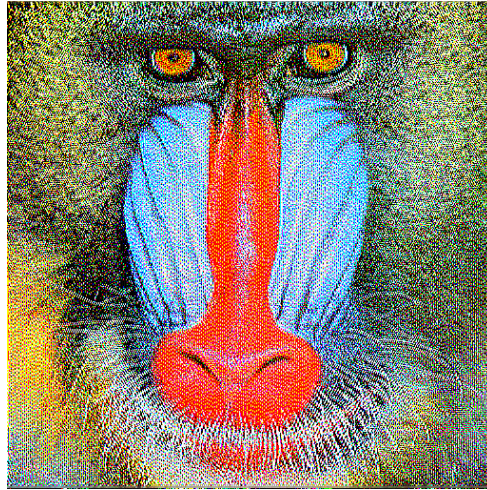


Imagem 4: difusão de Stevenson-Arce na banda de cor

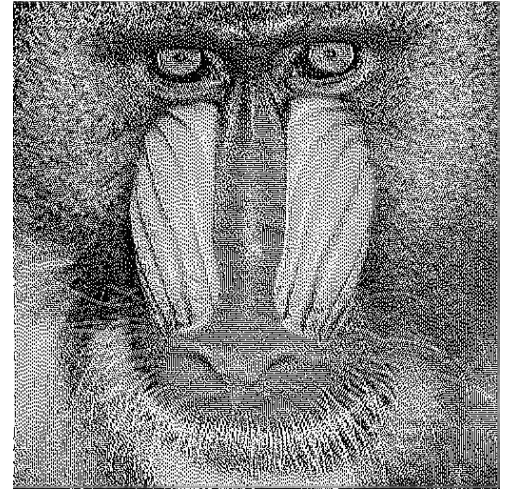


Imagem 5: difusão de Stevenson-Arce na banda monocromática

3.3.3. Burkes

Através da observação, deduz-se que esta técnica funciona muito bem para imagens coloridas, conseguindo manter os tons vivos. Porém, para imagens monocromáticas, observou-se a presença de estrias pela imagem, assim, surge a hipótese de que a formação das estrias ocorre por conta do modo como a imagem é percorrida (zigue-zague).

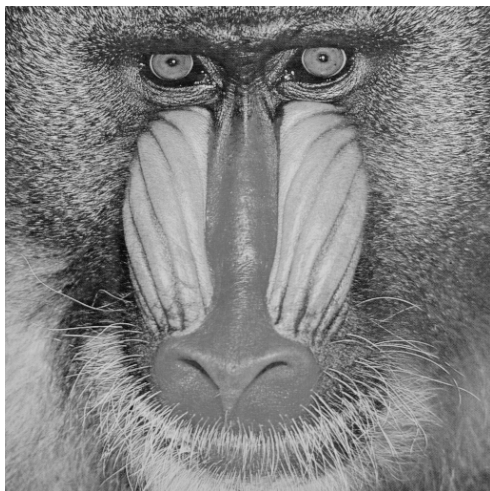


Imagem 1: imagem original (512x512)

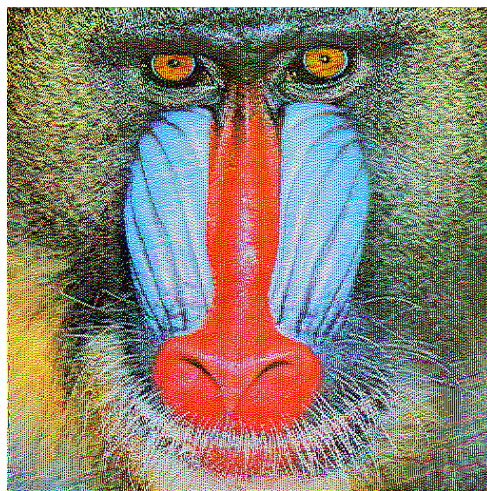


Imagem 6: difusão de Burkes na banda de cor

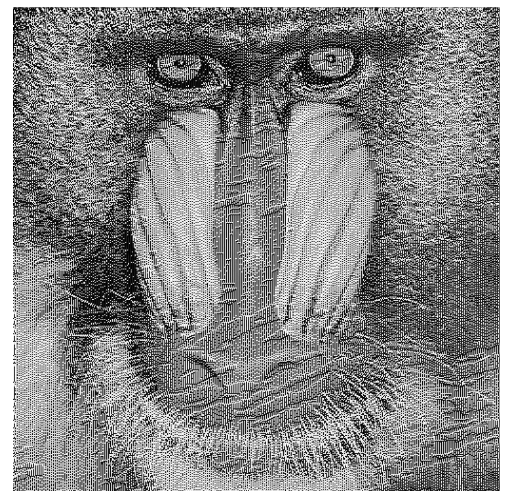


Imagem 7: difusão de Burkes na banda monocromática

3.3.4. Sierra

Aparentemente, um filtro que funciona bem tanto para imagens coloridas quanto para monocromáticas.

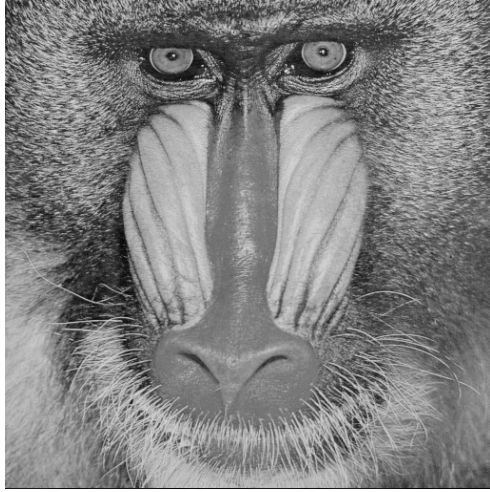


Imagem 1: imagem original (512x512)

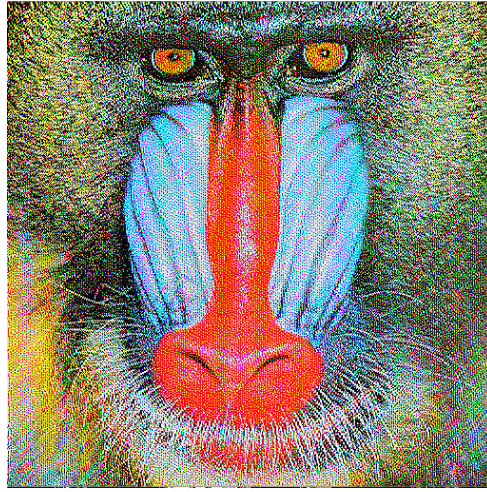


Imagem 8: difusão de Sierra na banda de cor

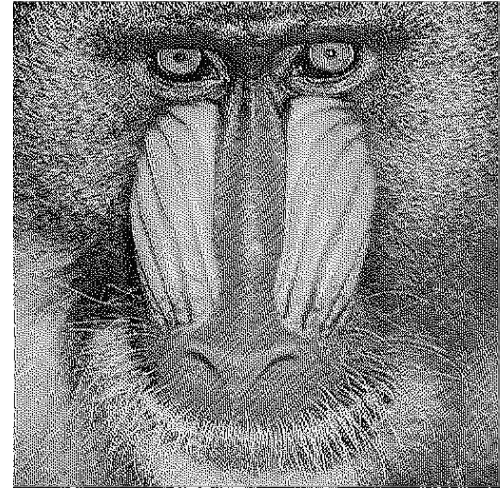


Imagem 9: difusão de Sierra na banda monocromática

3.3.5. Stucki

Este filtro consegue manter muito bem a qualidade da imagem original, porém agrega a percepção de uma imagem granulada ou então com alta nitidez.

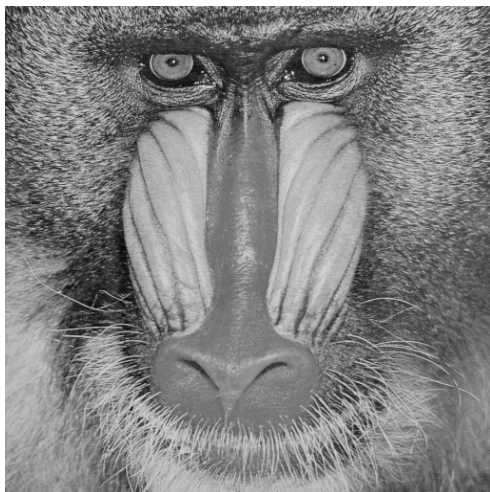


Imagem 1: imagem original (512x512)

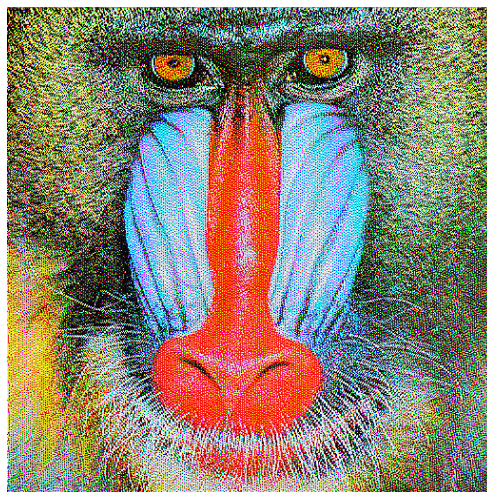


Imagem 8: difusão de Sierra na banda de cor

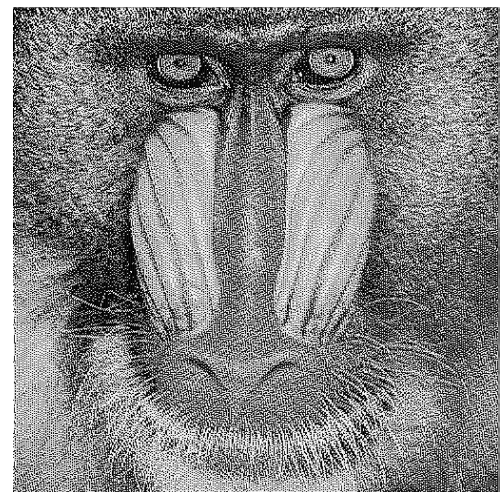


Imagem 9: difusão de Sierra na banda monocromática

3.3.6. Jarvis, Judice e Ninke

Produziu-se uma imagem com um aspecto altamente granulado, porém a colorida ainda consegue manter as cores bem parecidas com as originais.

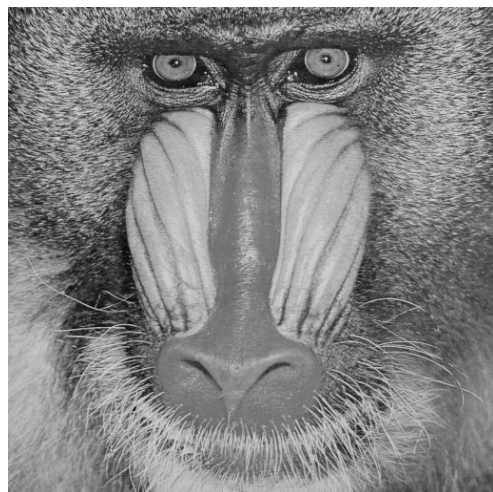


Imagem 1: imagem original (512x512)

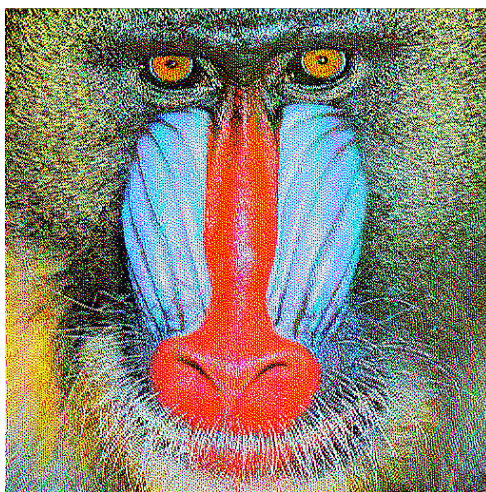


Imagem 8: difusão de Sierra na banda de cor

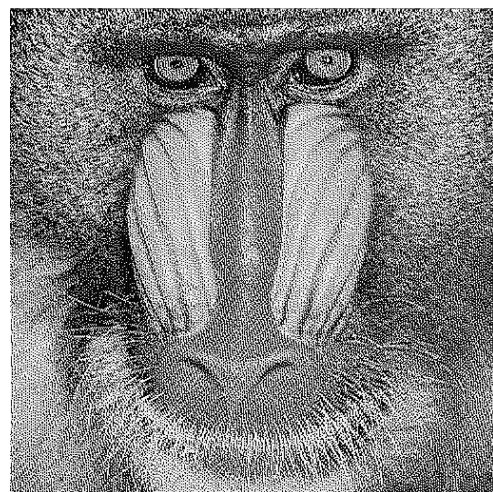


Imagem 9: difusão de Sierra na banda monocromática

Agora, para uma análise dos padrões e imagens finais que resultam da aplicação dos filtros de difusão de erro, fez-se quadrados de lado 80 (oitenta) cujos centros estão localizados no centro da imagem. Obteve-se assim os seguintes resultados.



Imagem 10: imagem fonte

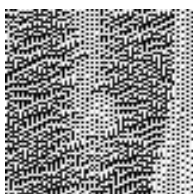


Imagem 11: Floyd e Steinberg



Imagem 12: Stevenson e Arce

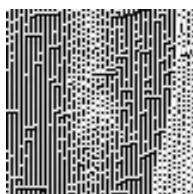


Imagem 13: Burkes



Imagem 14: Sierra

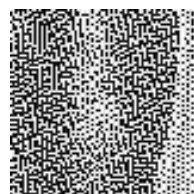


Imagem 15: Stucki



Imagem 16: Jarvis, Judice e Ninke



Imagem 17: imagem fonte

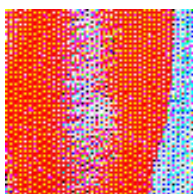


Imagem 18: Floyd e Steinberg

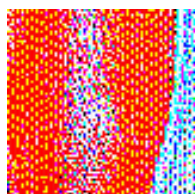


Imagem 19: Stevenson e Arce

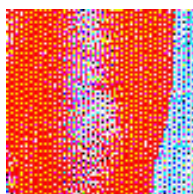


Imagem 20: Burkes

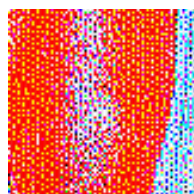


Imagem 21: Sierra

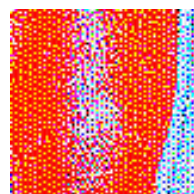


Imagem 22: Stucki

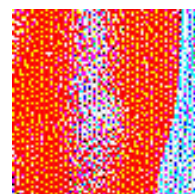


Imagem 23: Jarvis, Judice e Ninke

3.4. frequency_domain.py

Neste arquivo estão presentes algumas funções que compõem a realização da filtragem no domínio de frequência. Para a criação das máscaras para cada filtro, criou-se uma função *masking(mask_type: int, height: int, width: int, r1: int, r2: int = None)* que recebe o tipo da máscara um valor {0: *low pass filter*, 1: *high pass filter*, 2: *band pass filter*, 3: *band reject filter*}, a altura, a largura e os raios.

A função que gera a inversa da transformada discreta é a *inverseFDT(img: np.ndarray)*, que recebe a imagem já filtrada. E a que performa a criação dos núcleos é a *frequencyDomainFiltering(img: np.ndarray)*, abordar-se-á mais sobre esta abaixo:

3.4.1. Espectro de Fourier

O espectro de Fourier é calculado através da magnitude da imagem depois que passou pela transformada discreta e a centralização do espectro de frequência. Assim, obtém-se o seguinte:

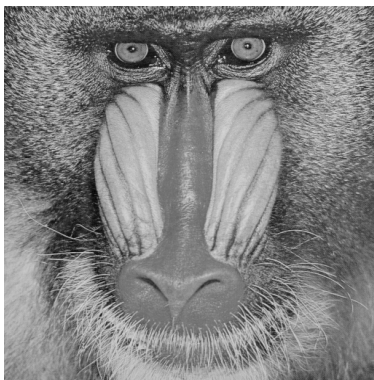


Imagem 1: imagem original
(512x512)

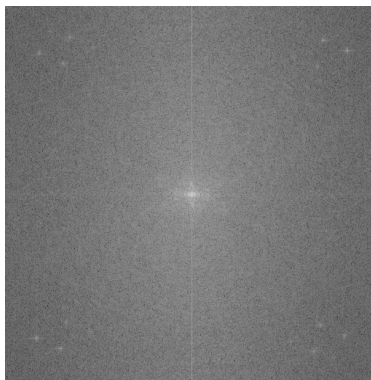


Imagem 24: espectro de
Fourier

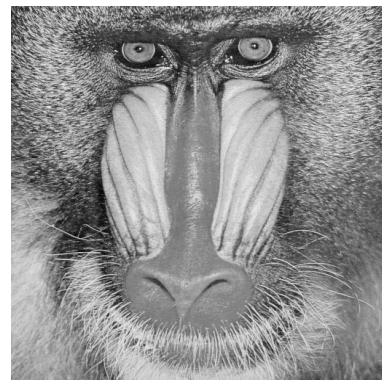


Imagem 25: aplicação da
inversa de Fourier

3.4.2. Aplicação dos filtros

Aplicou-se assim os quatro tipos de filtro: passa-baixa, passa-alta, passa-faixa e rejeita-faixa. Vale ressaltar que os filtros aplicados são todos ideais. Assim, obteve-se as seguintes máscaras e núcleos aplicados :

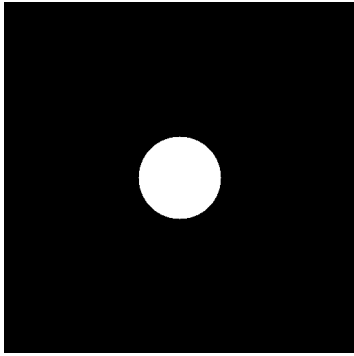


Imagem 26: Máscara passa-baixa com $r1 = 60$

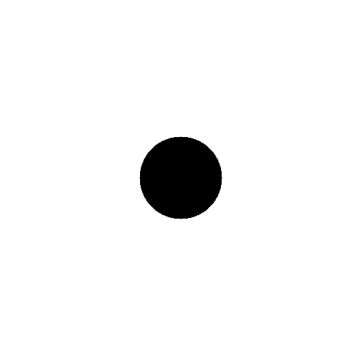


Imagem 27: Máscara passa-alta com $r1 = 60$

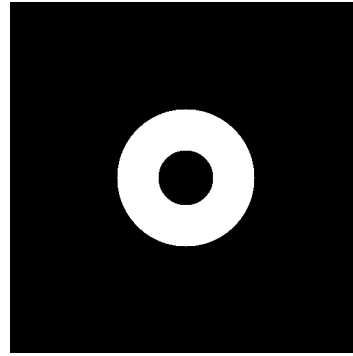


Imagem 28: Máscara passa-faixa com $r1 = 40$ e $r2 = 100$

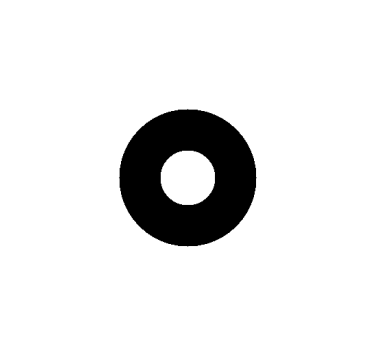


Imagem 29: Máscara rejeita-faixa com $r1 = 40$ e $r2 = 100$

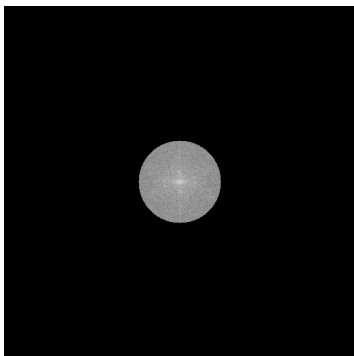


Imagem 30: Núcleo do passa-baixa aplicado na magnitude, $r1 = 60$

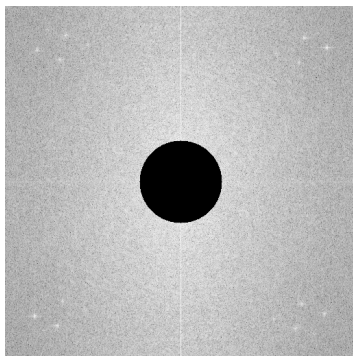


Imagem 31: Núcleo do passa-alta aplicado na magnitude, $r1 = 60$

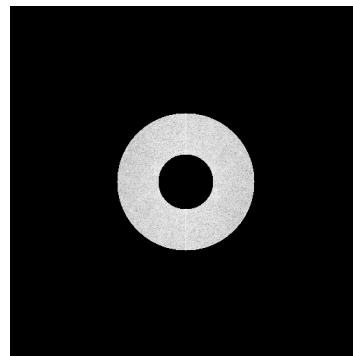


Imagem 32: Núcleo do passa-faixa aplicado na magnitude, $r1 = 40$ e $r2 = 100$

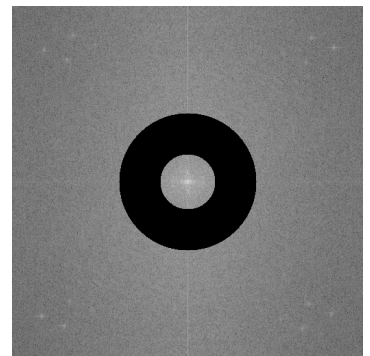


Imagem 33: Núcleo do rejeita-faixa aplicado na magnitude, $r1 = 40$ e $r2 = 100$

Agora, pode-se obter a imagem filtrada através de $mask * f_shift_img$ e a inversa desse resultado. Assim, tem-se as seguintes imagens filtradas.

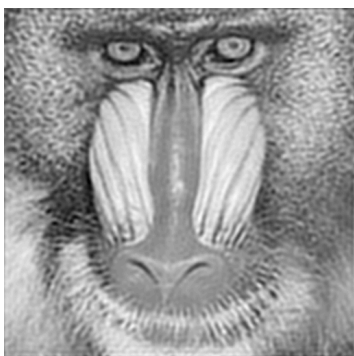


Imagem 34: Imagem com filtro passa-baixa

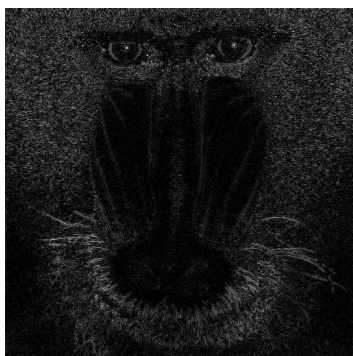


Imagem 35: Imagem com filtro passa-alta

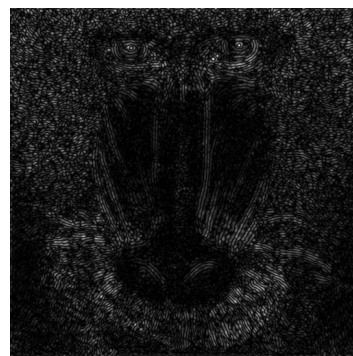


Imagem 36: Imagem com filtro passa-faixa

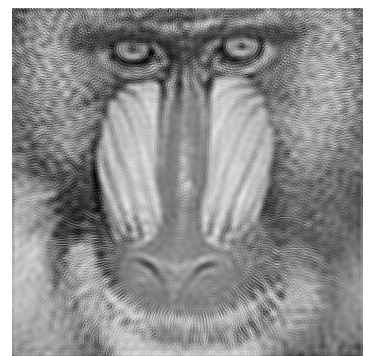


Imagem 37: Imagem com filtro rejeita-faixa

Percebe-se com a Imagem 34 que ocorre uma suavização das altas frequências, como bordas e transições mais abruptas de cores. Com a Imagem

35, percebe-se o destacamento das bordas e transições abruptas de cores. Já com a Imagem 36, onde foi aplicado um filtro de passa-faixa, permite-se que somente as frequências contidas na seção circular de raio menor 40 e raio maior 100 passem, isto é, somente essas frequências ficarão aparentes. Nota-se também que este filtro assemelha-se muito ao anterior, ao menos na configuração apresentada. Por fim, tem-se a aplicação do filtro rejeita faixa, que atenua as frequências na faixa especificada em detrimento das outras, proporcionando um aspecto de suavização da imagem, porém de uma maneira menos singela que o filtro de passa-baixa.

A fim de conseguir outros resultados realizou-se as mesmas aplicações para uma outra imagem colorida que foi transformada para uma escala de cinza, como pode ser analisado abaixo.



Imagem 38: peppers.png
(imagem fonte colorida)

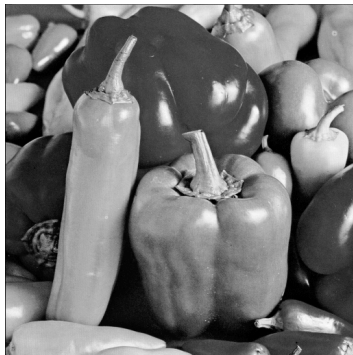


Imagem 39:
peppers_mono.png



Imagem 40: Espectro de
Fourier (magnitude)



Imagem 41: Inversa de
Fourier

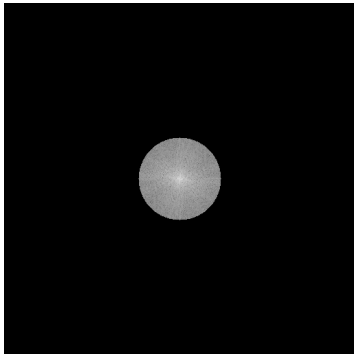


Imagem 42: núcleo
passa-baixa

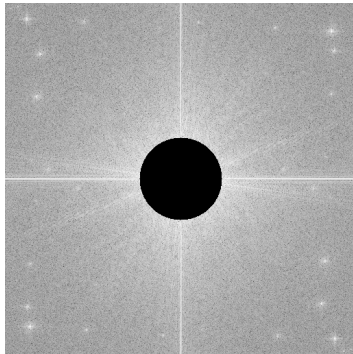


Imagem 43: núcleo passa-alta

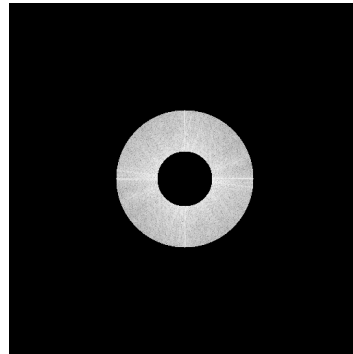


Imagem 44: núcleo
passa-faixa

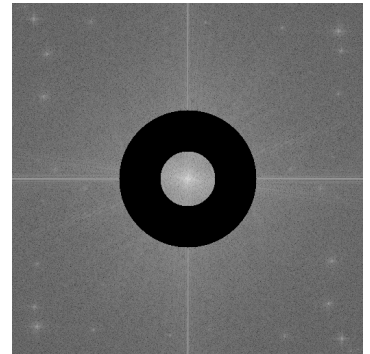


Imagem 45: núcleo
rejeita-faixa



Imagem 46: Imagem com filtro passa-baixa

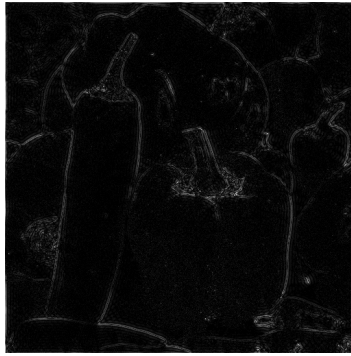


Imagem 47: Imagem com filtro passa-alta

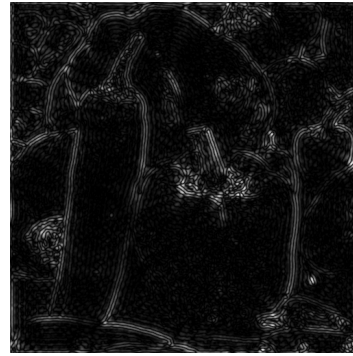


Imagem 48: Imagem com filtro passa-faixa



Imagem 49: Imagem com filtro rejeita-faixa

3.5. `compression.py`

A compressão é obtida através do cálculo *SVD* (*Singular Value Decomposition*) que gera três componentes um de vetores singulares esquerdos (u), um de valores singulares (d) e um de vetores singulares direitos (v). A partir disso, obtém-se uma imagem com dimensionalidade menor que preserva suas características mais importantes com base em k , isto é, o número de interações. Assim, por fim reconstrói-se a imagem comprimida com base nas três componentes que passaram por um *slicing* em relação a k .

Assim, para avaliar diferentes resultados considerou-se três k (10, 90, 150), de modo a ter três imagens comprimidas como pode ser conferido abaixo:

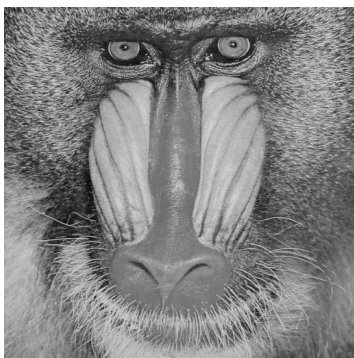


Imagem 1: imagem original (512x512)

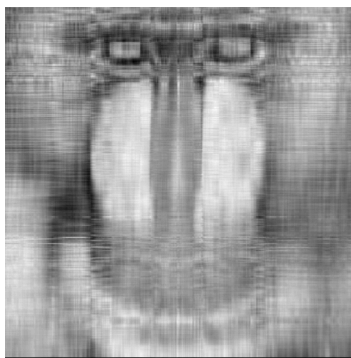


Imagem 50: $k = 10$

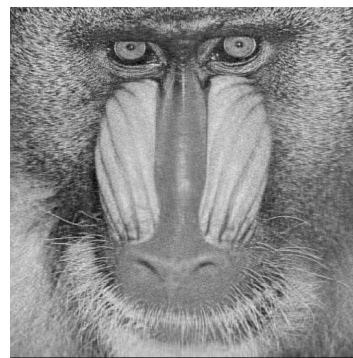


Imagem 51: $k = 90$

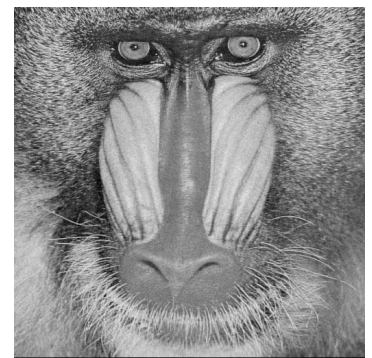


Imagem 52: $k = 150$

Nota-se que quanto maior o k , maior a compressão da imagem e consequentemente “mais leve” ela fica, visto que a imagem original (Imagem 1) tem 238.7 *kB* e as imagens 50, 51 e 52 têm 197.1 *kB*, 321.0 *kB* e 371.8 *kB*, respectivamente.

Analisando, agora, os histogramas de cada compressão, assim como, da imagem original observa-se que quanto maior o k é mais similar o histograma da compressão é do da imagem original.

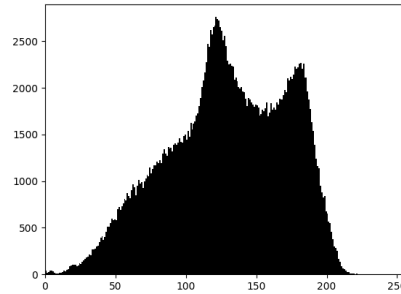


Imagem 53: histograma da imagem original

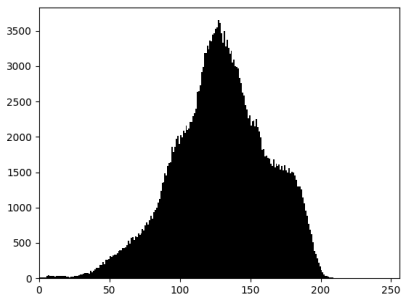


Imagem 54: histograma da imagem cujo $k = 10$

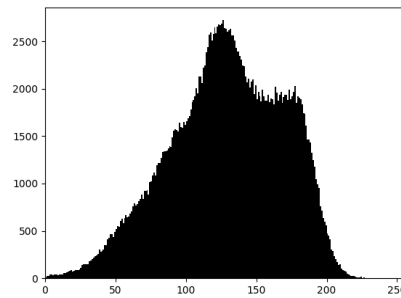


Imagem 55: histograma da imagem cujo $k = 90$

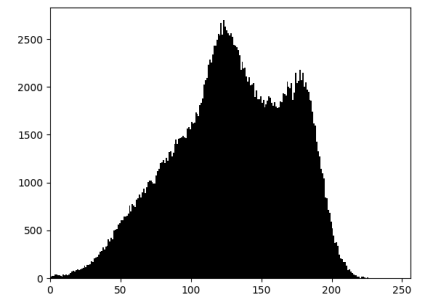


Imagem 56: histograma da imagem cujo $k = 150$